



BITS Pilani

Pipeline Hazards

Pipelined Processor

- Four types: Scalar Pipeline, Superscalar Pipeline, Superpipeline, Superpipeline Superscalar depending upon the following criteria with assumptions that all the pipelined processors are of k stages.
- Machine Pipeline Cycle (MPC): Time taken by each stage to process an instruction.
- Instruction Issue Rate (ISR): Number of instructions that can be issued simultaneously.
- Instruction Issue Latency (ISL): Time interval between issue of two instructions.
- Instruction Level Parallelism (ILP): Number of instructions that can be executed simultaneously in the pipeline.

Machine Type	Scalar Pipeline	Superscalar Pipeline	Super pipeline	Super pipeline Superscalar
MPC	1	1	$1/n$	$1/n$
ISR	1	m	1	m
ISL	1	1	$1/n$	$1/n$
ILP	1	m	n	mn

Figure : Parameters of Pipelined Processor.

Types of Pipelined Processors

- Scalar (base) pipeline - $MPC = 1$, $ISR = 1$, $ISL = 1$, $ILP = 1$; one instruction issued per cycle
- Superscalar - $MPC = 1$, $ISR = m$, $ISL = 1$, $ILP = m$; multiple instructions issued each cycle using parallel pipelines
- Superpipeline - $MPC = 1/n$, $ISR = 1$, $ISL = 1/n$, $ILP = n$; stages split into finer sub-stages clocked n times faster
- Superpipelined superscalar - $MPC = 1/n$, $ISR = m$, $ISL = 1/n$, $ILP = m \times n$; combines multiple issue with a faster, deeper pipeline for maximum parallelism



Pipeline Hazards

- Where one instruction cannot immediately follow another
- Types of hazards
 - Structural hazards - attempt to use the same resource by two or more instructions
 - Control hazards - attempt to make branching decisions before branch condition is evaluated
 - Data hazards - attempt to use data before it is ready
- Can always resolve hazards by waiting

- basic pipeline: single, in-order issue
- first extension: multiple issue (superscalar)
- second extension: scheduling instructions for more Instruction Level Parallelism (ILP)
- option #1: dynamic scheduling (by the hardware)
- option #2: static scheduling (by the compiler)

• Data Hazards



- When multiple instructions are in execution simultaneously (up to five in the 5-stage pipeline), data dependencies can cause data hazards, which must be resolved for correct operation.
- Note that dependencies are properties of the software only. Data hazards are caused by data dependencies and by the structure and instruction flow of the microarchitecture.

- Data Hazards

- **RAW(read after write)** A subsequent instruction needs the result of the current instruction. If the dependent instruction does not wait for the operand it may read a stale value (a RAW hazard).

Table 3.6 RAW hazards on registers after an ALU instruction

	Clock ==>	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	ADD R1,R2,R3	IF	ID	EX	ME	WB				
I2	ADDI R3,R1,#4		IF	ID	EX	ME	WB			
I3	LW R5,0(R1)			IF	ID	EX	ME	WB		
I4	ORI R6,R1,#20				IF	ID	EX	ME	WB	
I5	SUBI R1,R1,R7					IF	ID	EX	ME	WB

• Data Hazards

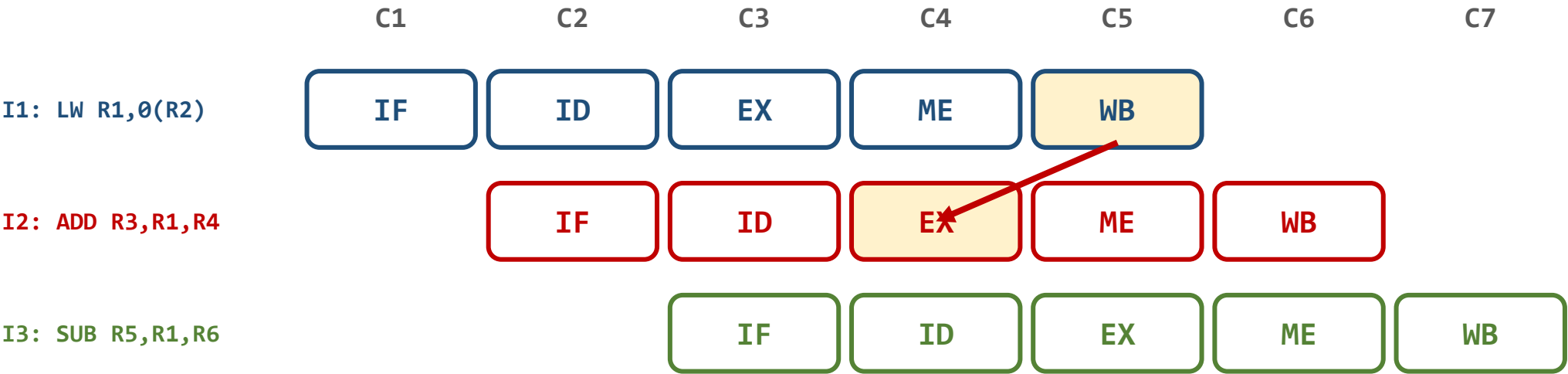


- **WAR (write after read)** A subsequent instruction writes to an operand which is read by the current instruction. If the write is able to race ahead of the read, the current instruction reads a future value instead of the current value (a WAR hazard).
- **WAW (write after write)** A subsequent instruction writes into the same operand as the current instruction. If the second write is allowed to race ahead of the first write, then the final value of the operand is the first (oldest) value and therefore is stale (a WAW hazard).
- Data dependencies on memory operands do not cause hazards in the 5-stage pipeline, because all loads and stores execute in the ME stage and reach the ME stage in process order.
- There cannot be any WAW or WAR hazards on registers.
- WAW hazards are impossible because all instructions modifying the register file reach the WB stage in process order. The same argument also holds for WAR hazards because instructions always reach the decode stage, where registers are read before any subsequent instruction in process order.

Data Hazards in Action

Click through the cycles: watch instructions advance and the RAW dependency collide.

RAW hazard on R1



I1 writes R1 in WB (cycle 5), but I2 needs R1 in EX (cycle 4) → forward EX/MEM → EX, or stall one cycle. I3 reads R1 in cycle 5 — also RAW.

Straight Line code

- RAW/ true

- Load- use

I1: load r1, a;

I2: add r2, r1, r1;

- Define –used

I1: mul r1, r4, r5

I2: add r2, r1, r1;

Straight Line code

- WAR /false/anti

I1: mul r1,r2,r3

I2: add r2,r4,r5

Straight Line Code

- WAW/output

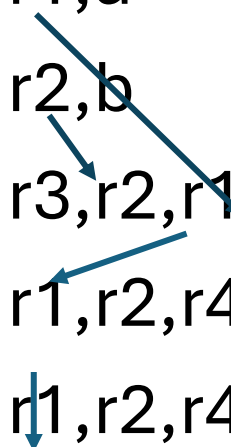
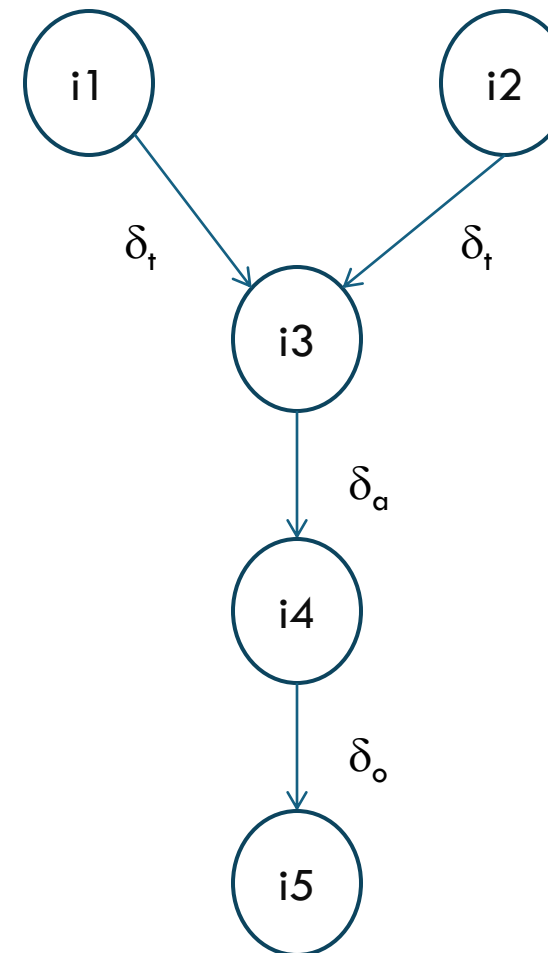
l1: mul r1,r2,r3

l2: add r1,r4,r5

Data dependency graph

```

load  r1,a
load  r2,b
add   r3,r2,r1
mul   r1,r2,r4
div   r1,r2,r4
  
```

- load r1,45(r2) (1)
- add r7 $\Downarrow$ r1, r5 (2)
- sub r8 $\Downarrow$ r1, r6 (3)
- or r9 $\Downarrow$ r5, r1 (4)
- brneq r7, target (5)
- add r10 $\Downarrow$ r8, r5 (6)
- xor r2 $\Downarrow$ r3, r4 (7)
-

(a) Identify each dependence; list the two instructions involved; identify which instruction is dependent; and, if there is one, name the storage location involved (register or memory).

- Inst (2) is dependent on inst (1) through r1
- Inst (3) is dependent on inst (1) through r1
- Inst (4) is dependent on inst (1) through r1
- Inst (5) is dependent on inst (2) through r7
- Inst (6) is dependent on inst (3) through r8
- Inst (6) is dependent on inst (5) through control
- Inst (7) is dependent on inst (5) through control

load r1, 45(r2) (1)
 add r7 $\Downarrow$ r1, r5 (2)
 sub r8 $\Downarrow$ r1, r6 (3)
 or r9 $\Downarrow$ r5, r1 (4)

brneq r7, target (5)

- Draw a pipeline diagram for the sequence, including stalls needed to rectify the hazards. Assumptions should be mentioned properly

add r10 $\Downarrow$ r8, r5 (6)
 xor r2 $\Downarrow$ r3, r4 (7)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
(1)	F	D	X	M	W											
(2)		F	D	d*	X	M	W									
(3)			F	p*	D	X	M	W								
(4)					F	D	X	M	W							
(5)						F	D	X	M	W						
(6)							F	D	s*							
(7)								F	s*							

In the above diagram, I assumed that the branch was predicted NT and was taken, just as an example to show the insertion of stall cycles.

- Structural Dependency or Resource Conflict

- Structural dependency is the result of resource conflict in the pipeline. When several instructions in the same cycle try to access the same resource, a resource conflict arises. A resource can be a register, memory, or ALU.

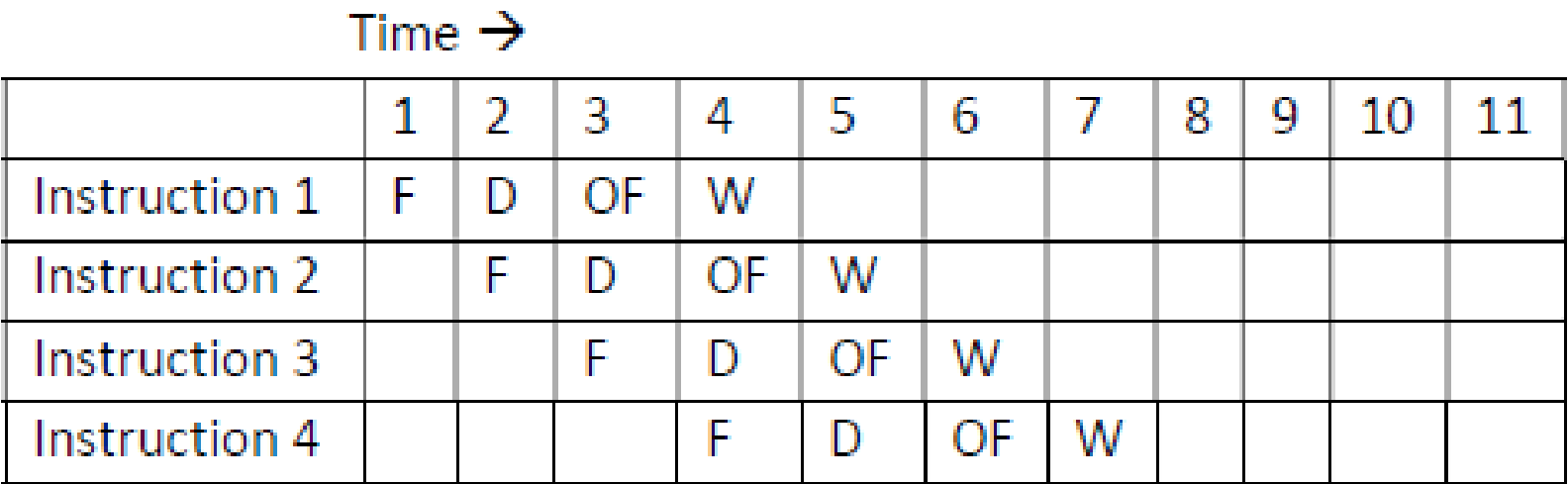
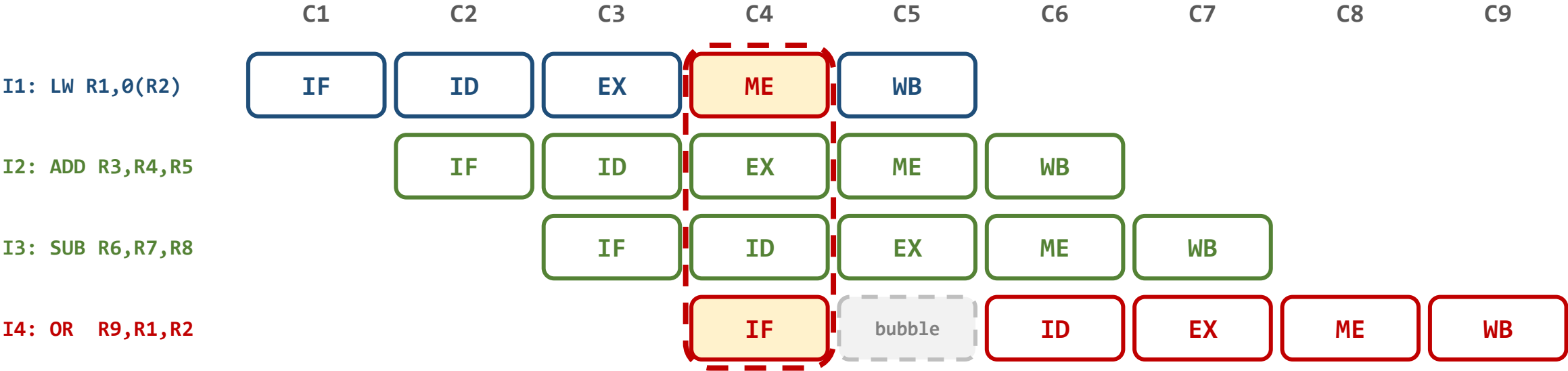


Figure : Timing diagram of a 4-Stage Instruction Pipeline

Structural Hazards in Action

Click through the cycles: two stages collide on a single memory port.

C4: one memory port, two requests



I4 stalls one cycle (bubble)

Structural hazard: two instructions need the same hardware in the same cycle — here I1’s ME access and I4’s IF both want memory in C4. Fix it by duplicating the resource (separate I-cache and D-cache, as in the Harvard-style 5-stage pipeline) or by stalling one instruction.

• Control Dependency



- A pipeline achieves its maximum utilization if all the stages of the
- pipeline take equal amount of time to process and there is no branch
- instruction in the program. However, if the program contains branch
- instruction, the pipeline suffers from branch penalty.

	Time →									← Branch				
Penalty →	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	F	D	O	E										
Instruction 2		F	D	O	E									
Instruction 3			F	D	O	E								
Instruction 4				F	D	O	E							
Instruction 5 (branch to instruction 25)					F	D	O	E						

Instruction 6						F	D	O						
Instruction 7							F	D						
Instruction 25								F	D	O	E			
Instruction 26									F	D	O	E		

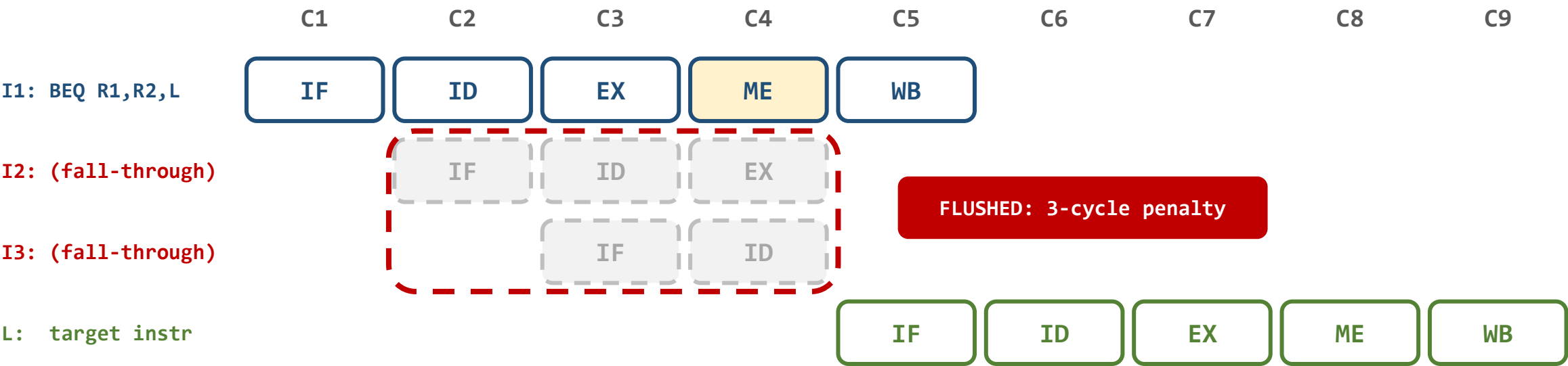
Figure : Timing diagram for Instruction Pipeline Operation

Control Dependencies

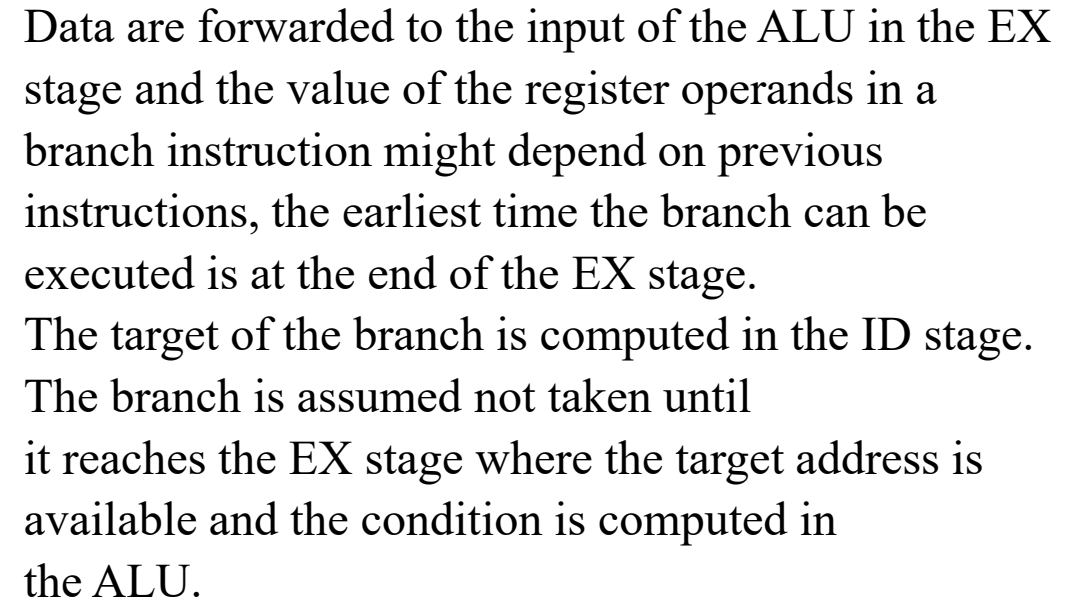
```
        mul    r1,r2,r3
        jz     zproc
        sub    r4,r7,r1
        .
        .
zproc:        load  r1,x
```

Control Hazards in Action

Click through the cycles: a taken branch squashes the wrong-path instructions.



Control hazard: the next instructions are fetched before the branch outcome is known. Resolving in ME costs 3 squashed instructions; resolving early in ID cuts the penalty to 1 cycle — or predict the branch and flush only on a mispredict.



BITS Pilani, Pilani Campus

Jump handling in the basic 5 stage of Pipeline



Jumps with absolute addresses could be executed at the end of IF since no address computation is needed. However, indirect jumps must be executed at the end of the decode stage because the register containing the target address must first be fetched. In this case, the instruction in IF must be flushed.

Out-of-order instruction completion



There are two possible types of data hazards on registers: RAW and WAW data hazards.

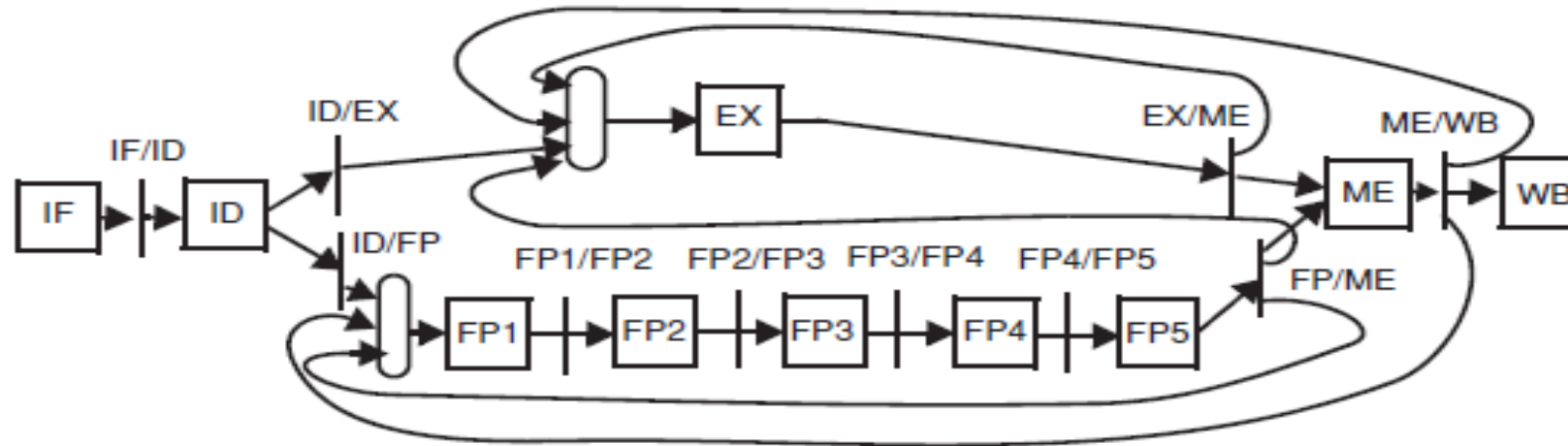


Figure 3.8. Pipeline with out-of-order execution completion.

The machine has two separate register files: one for floating-point operands and one for integer operands.

All instructions move through the ME and WB stages.

The execution of all floating-point instructions takes five cycles and is fully pipelined, and thus they may complete their execution after a subsequent integer instruction (ALU or load/store) in process order.

Hence, the major difference between this pipeline and the 5-stage pipeline is that instructions may complete their execution out of order.

Out-of-order instruction completion



Table 3.9 Increased RAW hazard stalls

Clock ==>		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11
I1	L.D F4, 0(R2)	IF	ID	EX	ME	WB						
I2	MULT.D F0,F4,F6		IF	ID	ID	FP1	FP2	FP3	FP4	FP5	ME	WB
I3	S.D F0, 0(R2)			IF	IF	ID	ID	ID	ID	ID	EX	ME

Table 3.10 Latency of operation and initiation interval

Functional units	Latency	Initiation interval
Integer ALU	0	1
Load	1	1
FP OP	4	1 (5 if not pipelined)

- The *latency of operation* of an instruction is the number of clocks that an immediately following dependent instruction in process order has to wait in ID to avoid a RAW hazard on one of its input register operands.
- The initiation interval is the minimum number of cycles between issuing two instructions of the same type to the execution units.

Structural Hazards



Table 3.11 Structural hazard on the write port of register file

	Clock ==>	C1	C2	C3	C4	C5	C6	C7	C8	C8	C9
I1	ADD.D F1,F2,F1	IF	ID	FP1	FP2	FP3	FP4	FP5	ME	WB	
I2	ADD.D F4,F2,F3		IF	ID	FP1	FP2	FP3	FP4	FP5	ME	WB
I3	L.D F10			IF	ID	EX	ME	WB			
I4	L.D F12				IF	ID	EX	ME	WB		
I5	L.D F14					IF	ID	EX	ME	WB	

In the WB stage, only one instruction can access each register write port in any clock. The WB stage just needs a separate bus for integer and FP operands.

Precise exceptions



Table 3.12 Problem with precise exceptions

	Clock ==>	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	ADD.S F1,F2,F1	IF	ID	FP1	FP2	FP3	FP4	FP5	ME	WB
I2	LW R2,0(R1)		IF	ID	EX	ME	WB			

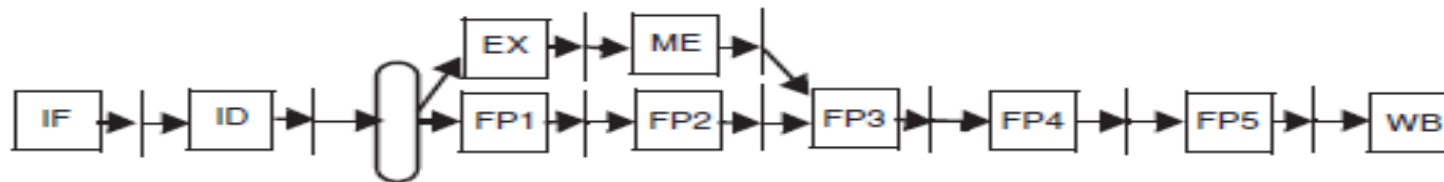


Figure 3.9. Forcing in-order completion in the WB stage.

- The hardware simply signals the software handler that an exception happened around some PC value.
- All potential exceptions must be detected as early as possible in the execution of an instruction. This approach may stifle pipelining.
- Finally, the architecture can be modified to force in-order traversal of the write-back stage in which exceptions are taken in process order, as in the 5-stage pipeline.

Dealing with precise exceptions



- In the 5-stage pipeline precise exceptions may be triggered in the IF stage (page fault), the ID stage (undefined instruction), the EX stage (arithmetic overflow), or the ME stage (page fault).
- A precise exception cannot happen in the WB stage because at that point the only remaining action is to write to the register file and only a hardware error could trigger an exception.

When a precise exception occurs in the 5-stage pipeline, the hardware must do the following:

- ✓ the faulting instruction plus all following instructions in process order must be squashed (i.e., the stage in which they are must be flushed);
- ✓ all instructions preceding the faulting instruction in process order must be completed; and
- ✓ the execution of the exception handler must start.

How to handle Exceptions?



- Complexity to handle exceptions
 - (1) the stages to flush vary with the stage in which the exception occurs,
 - (2) the exception condition and excepting PC must be accessible to treat the exception and reside in different stages,
 - (3) multiple exceptions may occur in the same clock cycle in different stages,
 - (4) exceptions must be taken in process order, not in temporal order.
- Solution: to flag an exception when it happens and keep the exception “silent”/ignore until the instruction reaches the WB stage.

Pipeline bubbles



- A bubble or a pipeline bubble represents a stage in the pipeline that cannot perform any useful operation due to the lack of data from previous stage of the pipeline.
- It is a method to prevent structural, data and branch hazards.
- Pipeline control logic analyzes if a hazard could arise while instructions are fetched.
- If this is the case, no operations (NOPs) are added to the pipeline by the control logic.
- As a result, before the next instruction runs, the previous one would have enough time to complete and avert the hazard.